

Advanced C++ Homework 2

學號 (請填寫), 姓名 (請填寫)

Floating Point 行為專案報告 —— 以 Kerr 黑洞測地線模擬引擎為例

報告情境。我要為團隊寫一個「黑洞影像」的科學計算程式：在旋轉黑洞 (Kerr 時空) 附近追蹤數十萬條光線，算出望遠鏡看到的影像，並與天文界實際使用的 [RAPTOR](#) (Bronzwaer et al. 2018, A&A 613, A2；為事件視界望遠鏡 EHT 產生合成影像的 ray tracer 之一) 比對。主管擔心「浮點數不安全」。本報告以這個真實問題為主軸：先說明我實作的引擎與驗證方式 (第 1 章)，再用它以及針對性的小實驗回答 Q0–Q7。所有程式碼、原始終端輸出與重現腳本都在 repository 中，每一個數據都可以在 `results/*.txt` 找到出處；截圖是這些輸出的畫面。

一句話結論：浮點數不是「不安全」，而是有規格、可預測、但不是實數。只要知道誤差從哪裡來 (捨入、抵銷、問題本身的病態性、編譯器/函式庫/硬體的差異)，就能量化並控制它 —— 本報告的引擎用 `double` 就能把黑洞陰影邊界算到與解析解相差 10^{-12} ，而 `-ffast-math`、`float16`、跨平台差異等陷阱也都能用實驗重現並說明原因。

- [Q0. 實驗環境](#)
- [1. 專案：Kerr 黑洞測地線模擬引擎](#)
 - [1.1 物理問題](#)
 - [1.2 我的實作 \(include/kerr/, header-only C++23\)](#)
 - [1.3 驗證一：與解析解比對 \(src/kerr_validate.cpp\)](#)
 - [1.4 驗證二：與 RAPTOR 逐條光線比對 \(src/raptor_compare.cpp\)](#)
 - [1.5 平行化技術比較 \(src/parallel_bench.cpp\)](#)
- [Q1. Benchmark：各種浮點型別的速度與精度](#)
 - [Q1.1 型別與格式 \(IEEE 754\)](#)
 - [Q1.2 浮點數的硬體實作](#)
 - [Q1.3 Benchmark 1：基本運算 \(experiments/q1_microbench.cpp\)](#)
 - [Q1.4 Benchmark 2：真實工作量 —— 同一個 Kerr 光線追跡 \(src/kerr_precision.cpp\)](#)
 - [Q1.5 記憶體與快取：float 真正的優勢在哪裡](#)
 - [Q1.6 小結：如何選型別](#)
- [Q2. Library Implementation：sin\(x\)](#)
 - [Q2.1 規格](#)
 - [Q2.2 glibc 2.39 的實作 \(sysdeps/ieee754/dbl-64/s_sin.c，源自 IBM Accurate Mathematical Library\)](#)
 - [Q2.3 實驗：精度與速度 \(experiments/q2_sin.cpp，以 MPFR 320 位元為正確值\)](#)
- [Q3. IEEE 754 與 -ffast-math](#)
 - [Q3.1 它做了什麼](#)
 - [Q3.2 範例：只因為 -ffast-math 而改變的結果 \(experiments/q3_fastmath.cpp\)](#)
 - [Q3.3 真實程式：Kerr 引擎與 RAPTOR](#)
 - [Q3.4 建議](#)
- [Q4. 浮點數輸出：printf\("%.9f"\) 是四捨五入嗎？](#)
- [Q5. \(加分\) 相同程式碼、相同編譯器版本、相同參數、無 UB，結果卻不同](#)

- 例一：硬體 (ISA) 不同 → FMA 自動融合 (contraction)
- 例二：ABI 不同 → long double 是不同的型別
- 例三：作業系統 / C 函式庫不同 → libm 不同
- 例四：同一個 binary、同一個 glibc，只是 CPU 不同
- 例五：核心數不同 → 平行歸約的結果不同
- Q6. 總結
 - Q6.1 我對浮點數運算的看法
 - Q6.2 額外細節：問題本身的「條件數」—— 再多位數也救不了病態問題
- Q7. AI 使用自我揭露

Q0. 實驗環境

所有實驗在同一台雲端 Linux 虛擬機上執行（Q5 另外使用 aarch64 交叉編譯 + QEMU 與 musl libc 模擬「不同環境」）。

項目	規格
CPU	Intel Xeon Processor @ 2.10 GHz (family 6, model 207 = 0xCF, Emerald Rapids / Raptor Cove 核心，stepping 2)，KVM 虛擬機 4 vCPU (1 thread/core)
指令集	SSE4.2, AVX2, FMA, AVX-512 (F/DQ/BW/VL/IFMA/VBMI…), AVX512-FP16, AVX512-BF16, AMX
快取	L1d 48 KiB/核, L1i 32 KiB/核, L2 2 MiB/核, L3 260 MiB (共享)
記憶體	15 GiB
OS / kernel	Ubuntu 24.04.4 LTS, Linux 6.18.44 (KVM guest)
C 函式庫	glibc 2.39 (Ubuntu 2.39-0ubuntu8.7)；Q5 另用 musl 1.2.4
編譯器	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)，-std=gnu++23；Q5 用同版本 aarch64-linux-gnu-gcc 13.3.0
其他	QEMU user 8.2.2 (執行 aarch64 binary)、Valgrind 3.22.0 (Cachegrind)、MPFR 4.2.1 (正確捨入參考值)、GSL 2.7.1 (RAPTOR 需要)、oneTBB 2021.11 (std::execution::par 後端)、Python 3.11 / NumPy 2.4 / Matplotlib 3.11、Node 22 + Playwright/Chromium (截圖與 PDF)
RAPTOR	github.com/tbronzwaer/raptor，固定 commit 08cb9a2 (2021-05-25)，原始碼未修改

編譯參數：每個實驗的參數明列在 Makefile (因為有些實驗本身就在研究參數)，預設為 g++ -std=gnu++23 -O2；benchmark 另外標示 -O3 -march=native、-ffast-math 等。

量測方法：std::chrono::steady_clock，每項量測重複 3 次取最小值；單執行緒量測時不跑其他工作。雲端 VM 的 turbo 頻率不可控，因此只比較相對數字；另外這台 VM 沒有開放硬體效能計數器 (perf stat 的 cache 事件回報 <not supported>)，所以快取命中率用 Valgrind Cachegrind 模擬、再以實測的 working-set sweep 佐證。

重現：make raptor && make all cross && ./scripts/run_all.sh (約 30 分鐘)，results/ 內即為本報告所有數據；python3 scripts/make_figures.py && node scripts/screenshots.js && python3 scripts/build_report.py 產生圖、截圖與本 PDF。



圖 0：實驗環境 (lscpu 節錄)

1. 專案：Kerr 黑洞測地線模擬引擎

1.1 物理問題

光在黑洞附近沿著零測地線 (null geodesic) 前進。從相機的每個像素往回追蹤一條光線：它可能掉進事件視界（黑色「陰影」）、打到吸積盤（發光）、或逃到無窮遠。旋轉黑洞由 Kerr 度規描述（單位 $G = c = M = 1$ ，自旋 a ）。

1.2 我的實作（[include/kerr/](#)，[header-only C++23](#)）

設計	說明
座標	ingoing Kerr–Schild (t, r, θ, ϕ) ：在事件視界上沒有座標奇異，光線可以平順穿越（Boyer–Lindquist 座標在視界處 $g_{rr} \rightarrow \infty$ ）。
方程式	Hamiltonian 形式 $H = \frac{1}{2} g^{ab}(x) p_a p_b = 0$ ， $\dot{x}^a = g^{ab} p_b$ ， $\dot{p}_a = -\partial H / \partial x^a$ 。KS 的逆度規只依賴 (r, θ) ，所以 p_t （能量）與 p_ϕ （角動量）在程式裡是逐位元守恒的；只剩 H 與 Carter 常數 Q 會漂移，拿來當精度診斷。RAPTOR 則是直接積分二階測地線方程，每步計算 64 個 Christoffel 符號。
型別	所有程式碼以 <code>template <class T></code> 撰寫，可實例化為 <code>float</code> 、 <code>double</code> 、 <code>long double</code> 、 <code>__float128</code> 、 <code>std::float16_t</code> ，甚至 <code>std::experimental::simd<double></code> （一次推進 8 條光線）。數學函式透過 <code>kerr::m::sin</code> 等多載統一（ <code>__float128</code> 對應 <code>libquadmath</code> 的 <code>sinq</code> ...）。
積分器	RK2（RAPTOR 預設）、RK4、Dormand–Prince 5(4)（自適應步長、FSAL）。步長規則與 RAPTOR 的 <code>stepsize()</code> 相同： $d\lambda = h / (\dot{r} /r + \dot{\theta} /\min(\theta, \pi - \theta) + \dot{\phi})$ 。
相機	與 RAPTOR 的 <code>initialize_photon()</code> 相同：像素 $\rightarrow$ 撞擊參數 $(\alpha, \beta) \rightarrow (E, L, Q)$ ，在 $r = 10^4 M$ 由 Boyer–Lindquist 轉到 KS。
場景	幾何薄、光學厚的 Keplerian 吸積盤（ $r_{\text{ISCO}} \sim 20 M$ ），觀測強度 $I = g^4 \cdot r^{-3} (1 - \sqrt{r_{\text{in}}/r})$ ， g 為重力紅移 + 都卜勒因子。

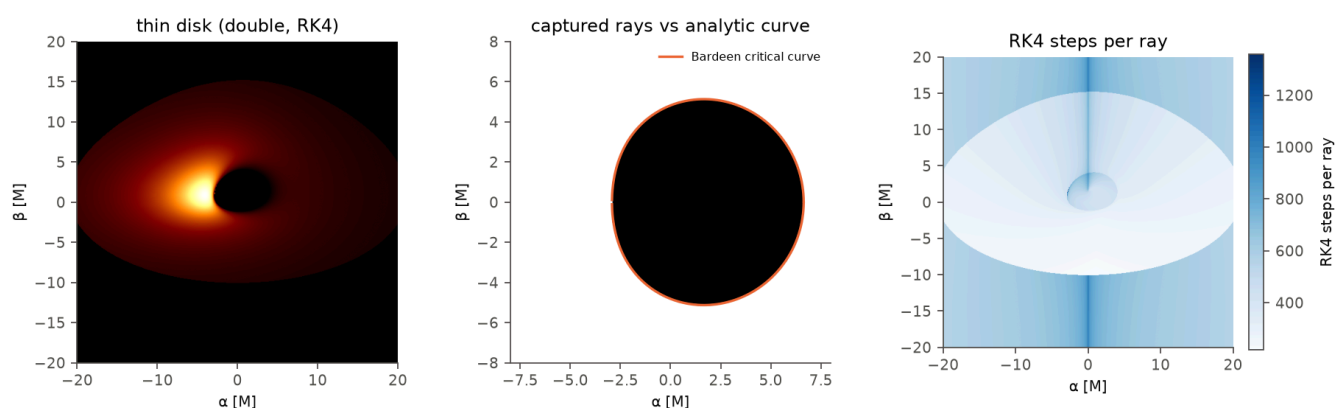


圖 1：左：本引擎產生的影像（ $a = 0.9375$ 、傾角 60° 、double、RK4）；中：被黑洞捕獲的光線（黑）與 Bardeen 解析陰影邊界（橘線）；右：每條光線的積分步數——貼近光子環的光線要多繞好幾圈，成本是一般光線的 2–6 倍（這在第 1.5 節的平行化中很重要）。

1.3 驗證一：與解析解比對（[src/kerr_validate.cpp](#)）

遠方觀測者看到的陰影邊界有解析解（Bardeen 1973 的 critical curve，由球面光子軌道給出）。我在 12 個方向上用二分法找「捕獲 $\leftrightarrow$ 逃逸」的交界，和以 `__float128` 計算的解析曲線比對：

```

bash — validate_shadow.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/kerr_validate --angles 12
Shadow edge vs Bardeen critical curve (a=0.9375, i=60 deg, 12 angles)
type      meth    tol/h  angles  rel. edge error: mean  max  |  time  steps
double    rk4      3e-02   12 | rel. edge error: mean 1.646e-09  max 2.937e-09 | 1.06 s 1361120 steps
double    rk4      1e-02   12 | rel. edge error: mean 1.936e-11  max 3.448e-11 | 1.63 s 4081056 steps
double    rk4      3e-03   12 | rel. edge error: mean 1.539e-13  max 2.808e-13 | 3.63 s 13614916 steps
float     dp45     1e-06   12 | rel. edge error: mean 1.223e-07  max 2.297e-07 | 0.84 s 112006 steps
float     dp45     1e-07   12 | rel. edge error: mean 8.292e-08  max 2.269e-07 | 0.84 s 162985 steps
double    dp45     1e-09   12 | rel. edge error: mean 4.093e-10  max 9.570e-10 | 1.05 s 507905 steps
double    dp45     1e-12   12 | rel. edge error: mean 5.266e-13  max 1.040e-12 | 1.59 s 2032693 steps
long double dp45 1e-14   12 | rel. edge error: mean 4.947e-15  max 1.022e-14 | 9.06 s 5123461 steps
__float128 dp45 1e-15   12 | rel. edge error: mean 5.153e-16  max 1.016e-15 | 229.56 s 8111855 steps

```

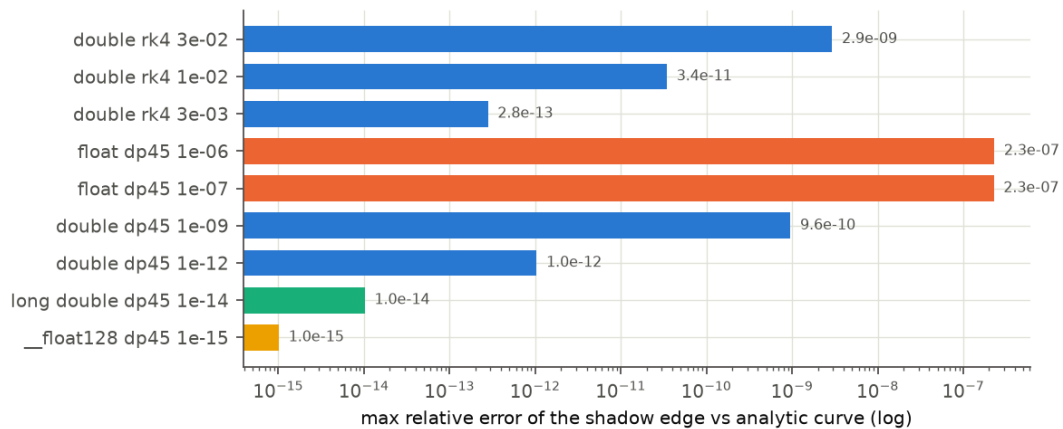


圖 2：陰影邊界的最大相對誤差。RK4 的誤差隨步長 h 以 h^4 下降 ($3 \times 10^{-9} \rightarrow 3 \times 10^{-11} \rightarrow 3 \times 10^{-13}$ ，每次 h 縮小 3 倍、誤差縮小 $\approx 81 = 3^4$ 倍)，證明方程式與積分器都正確；自適應 DP45 的極限則由型別決定：float $\approx 2 \times 10^{-7}$ ($\approx 2 \epsilon_{\text{float}}$)、double $\approx 10^{-12}$ 、long double $\approx 10^{-14}$ 、__float128 $\approx 10^{-15}$ （這裡已經被「二分法輸入參數是 double」限制）。

這是整份報告最重要的一張圖：它說明「浮點數誤差」其實是可以量化的，而且在這個問題中，演算法的截斷誤差（步長）通常遠大於 double 的捨入誤差。

1.4 驗證二：與 RAPTOR 逐條光線比對 ([src/raptor_compare.cpp](#))

raptor_bridge/raptor_bridge.c 把 RAPTOR 的 .c 檔（不修改）一起 #include 成單一 translation unit，直接呼叫它的 initialize_photon()、stepsize()、rk4_step() / rk2_step()（RAPTOR 把全域變數定義在 header 裡，GCC ≥ 10 預設 -fno-common 會連結失敗，單一 TU 可以繞過；完整的 RAPTOR 程式則用 -fcommon 編譯）。對 100×100 條光線（RAPTOR 範例設定： $a = 0.9375$ 、 $i = 60^\circ$ 、視野 40 M、STEPSIZE = 0.03）分別用 RAPTOR、本引擎（同一種積分器、同一個步長規則）與參考解（本引擎、long double、DP45、rtol 10^{-13} ）追蹤，再把逃逸光線推到同一半徑 $r = 1.01 \times 10^4$ 比較方向：

```

bash — raptor_compare.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/raptor_compare --res 100 --method rk4 --step 0.03 --dump build/raptor_cmp_rk4.bin
RAPTOR vs this engine: a=0.9375 i=60 fov=40 res=100 method=rk4 STEPSIZE=0.03 margin=0.1
max rel. difference of initial (r,th,p_r,p_th,p_phi): 2.930e-14
      RAPTOR      engine      reference
captured rays      486      486      486
time [s] (1 thread) 3.659      1.348
mean steps / ray    638.0      638.0
ns / step           573.5      211.3
max |dE| (E=p_t)    9.349e-06  0.000e+00
max |dL| (L=p_phi)  2.305e-05  4.486e-15
max |dQ|/max(1,Q)   9.663e-06  2.547e-06
max |g^ab p_a p_b|  1.836e-04  4.159e-04
status agreement: RAPTOR-engine 100.0000% RAPTOR-ref 100.0000% engine-ref 100.0000%
escaping rays (9514): angle between final directions at r = 1.01e4 [rad]
  RAPTOR - reference median 9.423e-08 p99 5.997e-06 max 1.877e-03
  engine - reference median 8.450e-09 p99 2.516e-08 max 5.422e-06
  RAPTOR - engine median 8.586e-08 p99 5.963e-06 max 1.874e-03

user@hw2:~/NTHU-AdvCPP-HW2$ build/raptor_compare --res 100 --method rk4 --step 0.01

```

RK4, STEPSIZE 0.03	RAPTOR	本引擎
初始條件差異 (與 RAPTOR)	—	2.9×10^{-14} (只差捨入)
捕獲光線數 (參考解 486)	486	486
每步時間 (單執行緒)	574 ns	211 ns (2.7× 快: Hamiltonian 形式不需 64 個 Christoffel 符號)
能量 E 漂移 / 角動量 L 漂移	9.3×10^{-6} / 2.3×10^{-5}	0 / 4.5×10^{-15} (結構上守恒)
最終方向誤差 (中位數 / p99)	9.4×10^{-8} / 6.0×10^{-6} rad	8.5×10^{-9} / 2.5×10^{-8} rad

- STEPSIZE 0.01 時兩者都以 h^4 收斂 (誤差 $\approx 1/81$)，確認兩個程式解的是同一個問題。
- RAPTOR 的預設積分器其實是 RK2 (parameters.h : int_method (RK2))。在同一步長下 RAPTOR RK2 捕獲 499 條光線 (多了 13 條錯誤分類)，方向誤差中位數 4×10^{-3} rad、最大 2.5 rad；本引擎的 RK2 為 2.6×10^{-4} / 1.2×10^{-2} 。換句話說，對這個問題而言誤差的主要來源是積分方法與步長，不是 double 的精度。
- 最大誤差都出現在陰影邊緣附近 —— 那些光線繞光子環好幾圈，是混沌的 (見 Q6)。

```

bash — raptor_compare.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/raptor_compare --res 100 --method rk2 --step 0.03
RAPTOR vs this engine: a=0.9375 i=60 fov=40 res=100 method=rk2 STEPSIZE=0.03 margin=0.1
max rel. difference of initial (r,th,p_r,p_th,p_phi): 2.930e-14
      RAPTOR      engine      reference
captured rays      499      486      486
time [s] (1 thread) 1.930      0.717
mean steps / ray    639.0      638.0
ns / step           302.0      112.4
max |dE| (E=p_t)    1.052e-02  0.000e+00
max |dL| (L=p_phi)  4.015e-02  4.486e-15
max |dQ|/max(1,Q)   4.413e-02  2.394e-03
max |g^ab p_a p_b|  1.912e-01  8.391e-02
status agreement: RAPTOR-engine 99.8700% RAPTOR-ref 99.8700% engine-ref 100.0000%
escaping rays (9501): angle between final directions at r = 1.01e4 [rad]
  RAPTOR - reference median 4.131e-03 p99 1.173e-01 max 2.516e+00
  engine - reference median 2.605e-04 p99 9.066e-04 max 1.244e-02
  RAPTOR - engine median 4.377e-03 p99 1.179e-01 max 2.522e+00

user@hw2:~/NTHU-AdvCPP-HW2$ build/raptor_compare_ofast --res 100 --method rk4 --step 0.03

```

1.5 平行化技術比較 (src/parallel_bench.cpp)

同一張 256×256 影像 (double、RK4、含吸積盤)，以不同平行技術渲染，並檢查結果與序列版逐像素一致 (mismatch 欄)。機器有 4 核。

```
bash — parallel.txt

double, 256x256 rays, 4 threads (hardware_concurrency=4), best of 3
technique          time [s]  speedup efficiency  imbalance status!= max rel dI  note
serial             6.392    1.00    100%      1.00      0    0.0e+00
std::thread static block  2.210    2.89    72%      1.36      0    0.0e+00
std::thread static cyclic rows  1.706    3.75    94%      1.03      0    0.0e+00
std::thread dynamic (atomic, 16)  1.658    3.85    96%      1.00      0    0.0e+00
OpenMP schedule(static)  2.192    2.92    73%      1.37      0    0.0e+00
OpenMP schedule(static,1)  1.636    3.91    98%      1.01      0    0.0e+00
OpenMP schedule(dynamic,16)  1.587    4.03   101%      1.00      0    0.0e+00
OpenMP schedule(guided)  2.211    2.89    72%      1.33      0    0.0e+00
std::for_each(execution::par)  1.582    4.04   101%      1.00      0    0.0e+00  TBB, busy time n/a
SIMD packets (1 thread)  1.391    4.60   460%      1.00      0    4.9e-14  lanes=8, lane utilisation 97%
SIMD packets + OpenMP dynamic  0.344   18.55  464%      1.00      0    4.9e-14
steps per ray: min 219 median 565 p99 816 max 1366

False sharing, 4 threads x 20000000 relaxed atomic increments on per-thread counters
packed (adjacent, share one 64-byte line): 1.444 s (72.20 ns/inc)
padded (alignas(64), one line per counter): 0.143 s (7.17 ns/inc) -> 10.1x faster
```

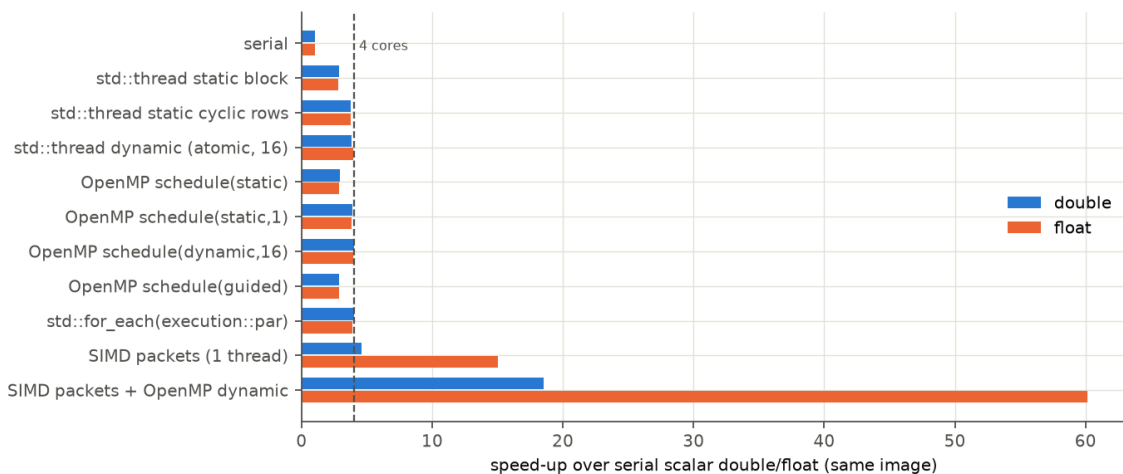


圖 3：相對於單執行緒 scalar 版本的加速 (藍：double，橘：float)。虛線為 4 核的理想值。

觀察與解釋：

1. 負載不平衡決定了排程策略的好壞。每條光線的步數從 219 到 1366 不等 (圖 1 右)，而且昂貴的光線集中在影像中央。把影像切成 4 個連續區塊 (std::thread static block、OpenMP schedule(static)) 時，拿到中央區塊的執行緒最慢，imbalance (最忙執行緒時間 / 平均) = 1.36，效率只有 72–73%。schedule(guided) 一開始就分大塊，同樣吃虧 (72%)。
2. 交錯分配 (cyclic rows、schedule(static,1)) 或動態排程 (atomic 工作計數器、schedule(dynamic,16)、TBB work stealing 的 std::for_each(std::execution::par, ...)) 都能達到 94–100% 效率 (量測雜訊使少數數字略超過 100%)：前者把昂貴區域平均分給每個執行緒，後者讓先做完的執行緒去拿新工作。
3. SIMD 是另一個維度：std::experimental::simd<double> (AVX-512, 8 lanes) 把 8 條相鄰光線綁在一起、以遮罩 (mask) 凍結已結束的 lane。單執行緒就有 4.6× (lane 使用率 97%：相鄰像素的步數很接近)，再加上 OpenMP 4 核達到 18.6×，而且結果與 scalar 版本的差異只有 5×10^{-14} (向量版 sin/cos 與 glibc 的捨入不同)。
4. float vs double (Q1 的伏筆)：scalar 程式中 float 並沒有比較快 (這個 -march=native build 中 float 甚至慢 27%，見 Q1.4)；但在 SIMD 版本中 float 一個暫存器可以放 16 個 lane：float SIMD 單執行緒是

scalar float 的 15×、是 double SIMD 的 2.6×，加上 4 核達到 60×，捕獲/逃逸分類完全相同，強度差異 1.7×10^{-4} (float 的精度)。

5. False sharing：4 個執行緒各自遞增「自己的」計數器，但計數器相鄰擺放、落在同一條 64-byte cache line 上：每次寫入都要搶這條 line 的所有權 (MESI 協定)，慢了 10 倍 (72 ns vs 7.2 ns 一次遞增)；以 `alignas(64)` 讓每個計數器獨佔一條 line 就解決了。

Q1. Benchmark：各種浮點型別的速度與精度

Q1.1 型別與格式 (IEEE 754)

一個二進位浮點數 = $(-1)^s \times 1.f \times 2^{e-\text{bias}}$ 。「精度」由尾數位數 p 決定 (machine epsilon $\epsilon = 2^{1-p}$)，「範圍」由指數位數決定。

C++ 型別 (x86-64 Linux)	格式	位元 (符號/指數/尾數)	p	ϵ	最大值	十進位有效位數	硬體
<code>std::float16_t</code> / <code>_Float16</code>	binary16	1/5/10	11	9.8×10^{-4}	65504	~3	AVX512-FP16 (本機有)；否則只有轉換指令 F16C
<code>float</code>	binary32	1/8/23	24	1.2×10^{-7}	3.4×10^{38}	~7	SSE/AVX
<code>double</code>	binary64	1/11/52	53	2.2×10^{-16}	1.8×10^{308}	~16	SSE2/AVX
<code>long double</code>	x87 80-bit extended	1/15/64 (顯式整數位)	64	1.1×10^{-19}	1.2×10^{4932}	~19	x87 FPU (佔 16 bytes)
<code>__float128</code>	binary128	1/15/112	113	1.9×10^{-34}	1.2×10^{4932}	~34	無硬體，libgcc 軟體模擬

(注意：`long double` 的格式由 ABI 決定——x86-64 Linux 是 80-bit、aarch64 Linux 是 binary128、MSVC 是 64-bit，這在 Q5 會成為「結果不同」的原因。)

Q1.2 浮點數的硬體實作

(1) 資料路徑。現代 x86-64 的 `float/double` 運算在 SSE/AVX 暫存器 (xmm/ymm/zmm) 上進行：scalar 指令 (`addss/addsd`) 只用最低的 lane，packed 指令 (`vaddps/vaddpd`) 一次處理整個暫存器——512-bit 暫存器可放 16 個 `float` 或 8 個 `double`。本機的 Raptor Cove 核心有兩個 FMA 單元 (port 0/5) 及兩個「快速加法器」(port 1/5)：加法延遲約 2 cycles、乘法/FMA 約 4 cycles，`float` 與 `double` 的延遲相同，因為同一組電路以相同級數的 pipeline 處理兩種寬度。

(2) 捨入。加法器先對齊指數、相加得到比 p 位更多的結果，再依 guard / round / sticky 三個額外位元以及 MXCSR 暫存器中的 rounding mode (預設 round-to-nearest-even) 捨入回 p 位，並在 MXCSR 設定 sticky 的例外旗標 (inexact、underflow、overflow、invalid、divide-by-zero)。IEEE 754 要求 $+ - \times \div \sqrt{}$ 與 FMA 都是「正確捨入」：結果等於把精確值捨入一次——所以基本運算是完全可預測、可重現的。FMA ($a \times b + c$ 只捨入一次) 是最重要的新增運算。

(3) 除法與開根號 使用獨立的迭代式單元 (radix-16 SRT/digit recurrence)，延遲比乘法長很多且只能部分 pipeline；`float` 需要的位數少，所以除法/開根號是 `float` 比 `double` 快的少數 scalar 運算 (見下表 `div`：4.2 vs 5.2 ns)。

(4) Subnormal (非正規數) 在很多微架構上需要 microcode assist，一次運算可能慢數十倍 (Q6 實測慢 17 倍)；`-ffast-math` 的 FTZ/DAZ 就是為了避開它 (Q3)。

(5) x87 與 `long double`。x87 是 1980 年的 8087 協處理器架構：8 個 80-bit 暫存器組成堆疊，沒有 SIMD，每次載入/存回 80-bit 值都需要 `fld/fstp tbyte` (較慢)，超越函數 (`f sin`) 是 microcode。GCC 對 `long double` 產生 `fadd/faddp` 等 x87 指令 (見截圖)。

解釋：

1. scalar 的 float 與 double 延遲幾乎一樣 (add 0.71 vs 0.72 ns、mul 1.6 vs 1.5 ns)：同一組硬體 (上一節 (1))。只有 div/sqrt 因迭代位數不同，float 稍快。
2. 向量化時 float 是 double 的 2 倍：-O2 的 add 0.136 vs 0.202 ns、-march=native 0.057 vs 0.116 ns —— 一個暫存器裝的 float 數量是 double 的兩倍；sqrt 在兩種 build 都是 1.07 vs 2.16 ns。float 的速度優勢來自 SIMD 寬度與記憶體頻寬 (Q1.5)，不是單一運算比較快。
3. long double：加乘延遲接近 double (x87 本身不慢)，但無法向量化 (throughput 3.6 ns，比 double 慢 30 倍)，而且 sinl 的實作較貴 (35 ns，見 Q2)。
4. __float128：每個運算都是函式呼叫 + 整數模擬，加法 18 ns、除法 28 ns、sqrt 135 ns、sin 379 ns (比 double 慢 25–60 倍)。
5. float16_t：在 -O2 每個運算 13 ns (兩次轉換函式呼叫!)，比 double 慢 20 倍；開 -march=native 後有硬體支援，向量加法 0.035 ns/元素 (一個 zmm 放 32 個 half)，是所有型別中最快的 —— 但精度只有 3 位數 (下一節)。

Q1.4 Benchmark 2：真實工作量 —— 同一個 Kerr 光線追跡 (src/kerr_precision.cpp)

48×48 條光線 (含吸積盤，RK4)，單執行緒。參考解是同一個演算法用 __float128 執行，所以誤差欄位是純捨入誤差 (截斷誤差相同而抵銷)。

```
bash — q1_kerr_precision.txt

user@hw2:~/NTHU-AdvCPP-Hw2$ build/kerr_precision --res 48 --reps 3
Kerr ray tracing, a=0.9375 i=60, 48x48 rays with thin disk, RK4 STEPSIZE=0.03, single thread, best of 3
reference = identical algorithm in __float128; errors are therefore pure round-off (RAPTOR k_theta formula)
type      ns/step  ns/rhs  rays/s status | dI med  dI max  d flux | dir med  dir max | max|2H|  max dQ
              | diff | (rel)  (rel)  (rel) | [rad]   [rad] |         | (rel)
float      199.7    49.9  10483.80    0 | 9.00e-07 7.08e-05 8.20e-08 | 1.34e-06 2.02e-05 | 1.70e-03 9.31e-05
double     222.1    55.5  9423.18    0 | 1.79e-15 9.20e-14 0.00e+00 | 2.64e-15 4.34e-14 | 3.36e-05 8.65e-07
long double 1508.4   376.9  1387.73    0 | 8.60e-19 4.34e-17 0.00e+00 | 1.25e-18 2.57e-17 | 3.36e-05 8.65e-07
__float128 19704.7 4923.6  106.23    0 | 0.00e+00 0.00e+00 0.00e+00 | 0.00e+00 0.00e+00 | 3.36e-05 8.65e-07
float16_t  2431.8   486.4 411211.07 2304 | 0.00e+00 0.00e+00 1.00e+00 | 0.00e+00 0.00e+00 | 0.00e+00 0.00e+00
```

型別	ns/step	相對 double	影像強度誤差 (中位數 / 最大)	光線方向誤差 (中位數)	評語
float	200	0.90× 時間	$9.0 \times 10^{-7} / 7.1 \times 10^{-5}$	1.3×10^{-6} rad	scalar 程式幾乎不快；誤差 $\sim \epsilon_{\text{float}} \times \text{放大}$
double	222	1	$1.8 \times 10^{-15} / 9.2 \times 10^{-14}$	2.6×10^{-15} rad	捨入誤差遠小於截斷誤差 ($\sim 10^{-8}$)
long double	1508	6.8× 慢	8.6×10^{-19}	1.3×10^{-18} rad	x87，無 SIMD；sinl/cosl 慢
__float128	19705	89× 慢	0 (參考解)	0	軟體模擬
float16_t	—	—	全部 2304 條光線失敗	—	溢位：相機設定中 $(r^2 + a^2)^2 \approx 10^{16} \gg 65504 \rightarrow \infty \rightarrow \text{NaN}$

解釋：

- 為什麼 float 在這裡沒有變快？光線追跡是 scalar、相依鏈很長 (每一步依賴上一步)、每次 RHS 呼叫 sin/cos 的程式；如 Q1.3 所示，scalar float 與 double 的延遲相同，sinf 只比 sin 快一點。記憶體用量極小 (見 Q1.5 的 Cachegrind)，也沒有頻寬優勢。唯有像第 1.5 節那樣把資料排成 SIMD，float 才有 2 倍的空間。
- 編譯器也會讓 float 變慢：用 -march=native 編譯時，GCC 的 SLP 自動向量化把 8 個元素的 RK4 狀態陣列打包成向量，產生的 shuffle 反而拖慢程式：float 從 223 變成 276 ns/step (double 225 $\rightarrow$ 225)；

加上 `-fno-tree-vectorize` 後 float 220、double 179 ns/step。「float 比 double 快」並不是硬體的保證，而是取決於資料排列與編譯器。

- H 漂移 ($\max|2H|$) : double、long double、`_float128` 都是 3.4×10^{-5} —— 這是 RK4 的截斷誤差，和型別無關；float 為 1.7×10^{-3} ，捨入誤差已經超過截斷誤差。這告訴我們 double 對這個問題是「精度剛好過剩」：加更多位數 (long double、quad) 完全沒有幫助，卻付出 7–90 倍的時間。
- float16 的失敗是「範圍」問題不是「精度」問題：中間值 $(r^2+a^2)^2$ 在 $r = 10^4$ (甚至 $r = 100$) 時就超過 65504。選擇型別時除了有效位數，還要檢查中間運算的動態範圍。

Q1.5 記憶體與快取：float 真正的優勢在哪裡

(a) Working-set sweep (實機量測，`experiments/cache_sweep.cpp`)：對大小 4 KiB–1 GiB 的陣列做加總 (float vs double)，以及隨機指標追逐 (測 load-to-use 延遲)。

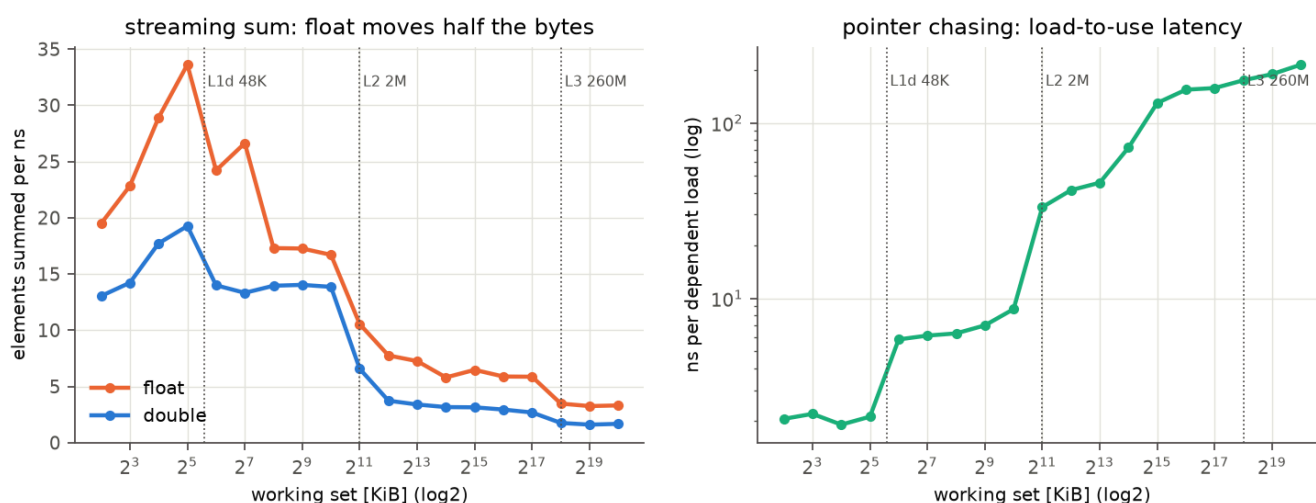


圖 6：左：每 ns 加總的元素數；右：相依載入延遲。虛線為 L1d/L2/L3 容量。

- 指標追逐清楚顯示階層：L1 ≈ 2 ns、L2 ≈ 6 –9 ns、L3 ≈ 33 –46 ns (2–8 MiB)；超過 ~16 MiB 後延遲逐步升到 150–215 ns (VM 實際可用的 L3 份額與 4 KiB 分頁的 TLB miss 都會貢獻)。
- 資料在 L1/L2 內時，加總受限於運算，float (16 lanes) 每 ns 處理約 1.5–1.9 倍於 double 的元素；超出快取後受限於記憶體頻寬 (兩者都約 13 GB/s)，float 因為只搬一半的位元組而處理 2 倍的元素 (3.3 vs 1.7 Gelem/s)。

(b) Cache 命中率 (Valgrind Cachegrind 模擬，`experiments/cache_patterns.cpp`)：沿列 (row-major, stride 1) 與沿行 (column-major, stride N) 加總 2048×2048 矩陣，以及光線追跡程式本身。

```
bash -- cachegrind.txt

user@hw2:~/NTU-AdvCPP-HM2$ build/cache_patterns row float 4096
row float n=4096 sum=45697936.0 time=0.016 s (0.94 ns/element)

user@hw2:~/NTU-AdvCPP-HM2$ build/cache_patterns row double 4096
row double n=4096 sum=50331645.0 time=0.024 s (1.41 ns/element)

user@hw2:~/NTU-AdvCPP-HM2$ build/cache_patterns col float 4096
col float n=4096 sum=45697936.0 time=0.164 s (9.76 ns/element)

user@hw2:~/NTU-AdvCPP-HM2$ build/cache_patterns col double 4096
col double n=4096 sum=50331645.0 time=0.288 s (12.40 ns/element)

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
row float n=2048 sum=12582907.0 time=0.042 s (9.90 ns/element)
==4122== D1 misses: 801,018 ( 274,523 rd + 526,495 wr)
==4127== D1 miss rate: 3.1% ( 5.8% + 2.5% )
==4127== L1d miss rate: 1.0% ( 0.2% + 1.2% )

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
row double n=2048 sum=12582907.0 time=0.049 s (11.68 ns/element)
==4132== D1 misses: 1,587,451 ( 536,668 rd + 1,050,783 wr)
==4137== D1 miss rate: 12.3% ( 11.4% + 2.8% )
==4137== L1d miss rate: 1.3% ( 0.2% + 1.4% )

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
col float n=2048 sum=12582907.0 time=0.204 s (48.59 ns/element)
==4137== D1 misses: 4,733,177 (4,286,682 rd + 526,495 wr)
==4137== L1d miss rate: 16.3% ( 89.3% + 2.5% )
==4137== L1d miss rate: 1.0% ( 0.2% + 1.2% )

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
col double n=2048 sum=12582907.0 time=0.289 s (47.63 ns/element)
==4145== D1 misses: 5,257,466 (4,286,683 rd + 1,050,783 wr)
==4145== L1d miss rate: 12.3% ( 89.3% + 2.8% )
==4145== L1d miss rate: 1.3% ( 0.2% + 1.4% )

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
type=float method=rk4 spin=0.9375 incl=60 res=24 time=1.821 s (316 rays/s, 6637.4 ns/step)
==4156== I1 miss rate: 0.00%
==4156== D1 misses: 15,693 ( 12,987 rd + 2,618 wr)
==4156== L1d miss rate: 0.0% ( 0.0% + 0.0% )
==4156== L1d miss rate: 0.0% ( 0.0% + 0.0% )

user@hw2:~/NTU-AdvCPP-HM2$ bash -c valgrind --tool=cachegrind --cache-sim=yes --cachegrind-out-file=/dev/null
type=double method=rk4 spin=0.9375 incl=60 res=24 time=3.201 s (180 rays/s, 11070.0 ns/step)
==4164== I1 miss rate: 0.00%
==4164== D1 misses: 15,693 ( 13,069 rd + 2,624 wr)
==4164== L1d miss rate: 0.0% ( 0.0% + 0.0% )
==4164== L1d miss rate: 0.0% ( 0.0% + 0.0% )

build/cache_patterns row float 2048 2>61 | grep -E 'n=D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
build/cache_patterns row double 2048 2>61 | grep -E 'n=D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
build/cache_patterns col float 2048 2>61 | grep -E 'n=D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
build/cache_patterns col double 2048 2>61 | grep -E 'n=D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
build/kerr_render_serial --type float --res 24 --disk 2>61 | grep -E 'type=I refs|I1 miss rate|D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
build/kerr_render_serial --type double --res 24 --disk 2>61 | grep -E 'type=I refs|I1 miss rate|D refs|D1 miss|L1d miss|D1 miss rate|L1d miss rate'
```

- 沿列走訪時，每條 64-byte cache line 含 16 個 float／8 個 double，D1 讀取 miss rate 5.8% 與 11.4%（≈ 1/16 與 1/8）（float 命中率較高，這就是「float 省頻寬」的微觀原因）；沿行走訪每次都碰新的一條 line，D1 讀取 miss rate 89%，實測（4096×4096）慢 9–10 倍。
- 光線追跡程式的 D1 miss rate 接近 0：每條光線的狀態只有 8 個數、幾十個暫存值，全部在暫存器與 L1 中——這是一個 compute-bound 程式，這也解釋了為何 Q1.4 中 float 沒有頻寬優勢。

Q1.6 小結：如何選型別

- 預設用 **double**：scalar 速度與 float 相同，精度多 9 位數，範圍 10³⁰⁸ 幾乎不會溢位。
- float** 只在「可量化或受記憶體頻寬限制」且誤差預算 ≥ 10⁻⁶ 時才划算（影像、圖學、ML、大陣列）。
- long double**/**__float128** 用來驗證（例如本報告的參考解），不要當作主要型別：慢 7–90 倍，而且 **long double** 在不同平台是不同格式。
- float16**/**bfloat16** 只適合儲存或容錯的 ML 計算：3 位有效數字、最大值 65504。
- 比「換更長的型別」更有效的通常是換更好的演算法（RK4 vs RK2、守恆形式的方程式、避免抵銷的公式）。

Q2. Library Implementation : `sin(x)`

Q2.1 規格

- C++ : `<cmath>` 的 `std::sin` 規格引用 C 標準 7.12.4.6 : 「sin 函數計算 x (以弧度為單位) 的正弦」。
C++ 提供 `float sin(float)`、`double sin(double)`、`long double sin(long double)` (以及 `sinf`、`sinl`) , 整數引數當作 `double` ; C++23 增加 extended floating-point 型別 (`std::float16_t` 等) 的多載, C++26 起為 `constexpr` (P1383) 。
- 精度 : C 標準 5.2.4.2.2 說 `<math.h>` 函數的精度是 implementation-defined —— 標準不要求正確捨入, 也不要求不同實作結果相同。IEEE 754-2019 §9.2 只「建議 (should)」sin 正確捨入。
- 特殊值 (C Annex F.10.1.6, glibc 遵守) : $\sin(\pm 0) = \pm 0$ (保留符號) ; $\sin(\pm\infty)$ 回傳 NaN 並觸發 `FE_INVALID` , 若 `math_errhandling & MATH_ERRNO` 則 `errno = EDOM` ; $\sin(\text{NaN}) = \text{NaN}$; 很小的非正規數回傳 x 本身並觸發 underflow。

```
bash — q2_sin.txt

Special values
sin(+0) = 0          errno=0      FE_INVALID=0 FE_UNDERFLOW=0 FE_INEXACT=0
sin(-0) = -0         errno=0      FE_INVALID=0 FE_UNDERFLOW=0 FE_INEXACT=0
sin(1e-310) = 1e-310 errno=0      FE_INVALID=0 FE_UNDERFLOW=1 FE_INEXACT=1
sin(+inf) = -nan      errno=EDOM  FE_INVALID=1 FE_UNDERFLOW=0 FE_INEXACT=0
sin(NaN) = nan        errno=0      FE_INVALID=0 FE_UNDERFLOW=0 FE_INEXACT=0
sin(1) = 0.841471     errno=0      FE_INVALID=0 FE_UNDERFLOW=0 FE_INEXACT=1
```

Q2.2 glibc 2.39 的實作 (`sysdeps/ieee754/dbl-64/s_sin.c` , 源自 IBM Accurate Mathematical Library)

x86-64 的 glibc 不使用 x87 的 `fsin` 指令, 完全以 SSE2 的 double 運算實作。依 $|x|$ 的高 32 位元分成 5 個區間 :

區間	做法
$ x < 2^{-26}$	直接回傳 x ($\sin x = x - x^3/6$, $x^3/6$ 小於半個 ulp)
$2^{-26} \leq x < 0.855469$	<code>do_sin(x, 0)</code> : 若 $ x < 0.126$ 用 Taylor 多項式 ; 否則查表 : 把 x 拆成 $x = x_i + dx$, x_i 是 $1/128$ 的倍數 (技巧 : <code>u.x = big + x </code> , <code>big = 1.5·2⁴⁵</code> , 加法的捨入自動把 x 四捨五入到 2^{-7} 的倍數, 低位元就是表格索引)。表 <code>__sincostab</code> 有 440 個 double, 存 $\sin(x_i)$ 、 $\cos(x_i)$ 以及它們的「尾數修正項」(double-double)。再用 $\sin(x_i+dx) = \sin x_i \cos dx + \cos x_i \sin dx$, dx 很小, $\sin dx$ 、 $\cos dx$ 只要 3 項多項式。
$0.855469 \leq x < 2.426265$	用 $\sin x = \cos(\pi/2 - x)$: $\pi/2$ 以兩個 double (<code>hp0 + hp1</code> , 共 106 位元) 表示, $t = hp0 - x $ 精確, 再呼叫 <code>do_cos(t, hp1)</code>
$2.426265 \leq x < 105414350$	Cody–Waite 範圍縮減 <code>reduce_sincos</code> : $n = \text{round}(x \cdot 2/\pi)$ (技巧 : 加上 <code>toint = 1.5·2⁵²</code> 讓 FPU 幫忙取整) ; $y = x - n \cdot (\pi/2)$, 其中 $\pi/2$ 拆成 4 段 (<code>mp1</code> 、 <code>mp2</code> 、 <code>pp3</code> 、 <code>pp4</code>) , 前幾段只有 20–30 個有效位元, 使 $n \cdot mp1$ 在 $n < 2^{27}$ 時精確 (無捨入) , 總共約 136 位元的 $\pi/2$, 得到 double-double 的 (<code>a</code> , <code>da</code>) ; 再依 $n \bmod 4$ 選 $\pm \sin/\pm \cos$
$ x \geq 105414350$	<code>__branred</code> (Payne–Hanek 式) : x 可能大到 10^{308} , $x \cdot (2/\pi)$ 需要 $2/\pi$ 的第 1000 多位才能得到正確的小數部分。表 <code>toverp</code> 存 $2/\pi$ 的二進位展開 (每段 24 位元) , 只取和 x 的指數對齊的那幾段相乘累加, 得到 $x \bmod \pi/2$
Inf / NaN	回傳 x/x (產生 NaN 與 <code>FE_INVALID</code>) , Inf 時設 <code>errno = EDOM</code>

其他實作細節 :

- 捨入模式：函式開頭 `SET_RESTORE_ROUND_53BIT(FE_TONEAREST)` —— 若呼叫者用 `fesetround` 改了捨入模式，函式內暫時切回 `round-to-nearest`，演算法的誤差分析才成立。
- 錯誤界：原始碼註解標示各區間 `max ULP 0.501–0.548`（「~0.55 ULP」）—— 不是正確捨入（正確捨入 ≤ 0.5 ULP）。
- 硬體 dispatch（IFUNC）：`sysdeps/x86_64/fpu/multiarch/s_sin.c` 把同一份原始碼編兩次：`__sin_sse2` 與以 `-mfma -mavx2` 編譯的 `__sin_fma`，執行期由 IFUNC resolver 依 CPU 選一個。以 `experiments/q2_ifunc.c` 印出 `sin` 解析後的位址並反組譯：本機預設選到含 36 個 `vfmadd*` 指令的版本；用 `GLIBC_TUNABLES=glibc.cpu.hwcaps=-AVX2,-FMA` 遮掉 FMA 後選到另一個位址、0 個 FMA 指令的版本（Q5 會看到這讓結果不同）。

```

bash — q2_sin.txt

user@hw2:~/NTHU-AdvCPP-HW2$ ./scripts/which_sin.sh build/q2_ifunc
sin resolved to 0x7fb79487f2d0 = /lib/x86_64-linux-gnu/libm.so.6 + 0x7b2d0 -> 36 FMA instructions in the first 1200 bytes (GLIBC_TUNABLES=<unset>)

user@hw2:~/NTHU-AdvCPP-HW2$ env GLIBC_TUNABLES=glibc.cpu.hwcaps=-AVX2,-FMA ./scripts/which_sin.sh build/q2_ifunc
sin resolved to 0x7f27e9a1b8c0 = /lib/x86_64-linux-gnu/libm.so.6 + 0x848c0 -> 0 FMA instructions in the first 1200 bytes (GLIBC_TUNABLES=glibc.cpu.hwcaps=-AVX2,-FMA)

```

Q2.3 實驗：精度與速度（`experiments/q2_sin.cpp`，以 MPFR 320 位元為正確值）

我另外寫了 60 行的 `mini_sin`（`fdlibm` 1993 的演算法：3 段 33 位元的 $\pi/2$ 做 Cody–Waite + 6 次 minimax 多項式），以及直接呼叫 `x87 fsin` 指令，與 `glibc` 比較。每個區間 20 萬個隨機輸入：

```

bash — q2_sin.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/q2_sin 200000
ULP error vs MPFR, 200000 random inputs per range (max ulp / % correctly rounded)
range (glibc branch) | glibc sin | mini_sin (fdlibm) | x87 fsin | glibc sinl | glibc sinf | libquadmath sinq
[2^-26, 0.855469) | 0.514 / 100.0% | 1.068 / 99.5% | 0.682 / 99.6% | 0.519 / 100.0% | 0.558 / 99.6% | 0.521 / 99.9%
[0.855469, 2.426265) | 0.503 / 99.9% | 1.098 / 90.1% | 0.584 / 97.8% | 0.521 / 99.9% | 0.544 / 99.8% | 0.552 / 99.7%
[2.426265, 105414350) | 0.515 / 99.9% | 1.1e+12 / 74.8% | 2.7e+10 / 12.6% | 1.404 / 97.3% | 0.558 / 98.7% | 1.126 / 97.3%
[105414350, 1e300] | 0.514 / 99.9% | inf / 0.0% | 1.2e+19 / 0.0% | 1.184 / 97.4% | 0.558 / 98.7% | 1.211 / 97.3%

Hard inputs (value printed with 17 significant digits, error in ulp)
x | sin(x) glibc | glibc | mini_sin | x87 fsin
3.1415926535897931 | 1.2246467991473532e-16 | 0.121 | 0.121 | 1.64e+11
6.2831853071795862 | -2.4492935982947064e-16 | 0.121 | 0.121 | 1.64e+11
355 | -3.0144353359488451e-05 | 0.384 | 0.384 | 135
103993 | -1.9129335778423752e-05 | 0.49 | 0.49 | 3.95e+04
1e+22 | -0.85220084976718879 | 0.0611 | 3.63e+87 | nan
5.3193726483265414e+255 | 1 | 9.89e-22 | nan | nan

Time per call [ns] (glibc sin, 1M calls per range, independent inputs)
[2^-26, 0.855469) | glibc 6.4 | mini_sin 3.1 | x87 fsin 38.0 | sinl 16.9
[0.855469, 2.426265) | glibc 6.4 | mini_sin 15.6 | x87 fsin 47.9 | sinl 173.9
[2.426265, 105414350) | glibc 22.8 | mini_sin 18.9 | x87 fsin 43.5 | sinl 177.9
[105414350, 1e300] | glibc 70.6 | mini_sin 7.8 | x87 fsin 25.9 | sinl 179.1

```

- `glibc sin` 在所有區間最大誤差 ≤ 0.515 ULP，99.9% 正確捨入，包括 x 到 10^{300} —— 這要歸功於 136 位元的 $\pi/2$ 與 Payne–Hanek。
- `x87 fsin` 在 $x = \pi$ 誤差 1.6×10^{11} ULP：Intel 的 `fsin` 內部只用 66 位元的 π ， x 接近 π 的倍數時 $x - k\pi$ 的抵銷把 π 的誤差放大 —— 這是 `glibc` 從 2.28 起完全不使用它的原因（Intel 在 2014 年修正了手冊中的精度宣稱）。
- `mini_sin` 在 $|x| < \pi/4$ 附近誤差約 1 ULP（沒有查表與 double-double 修正），且 $|x|$ 超過約 1.6×10^6 後完全錯誤（ 1.1×10^{12} ULP； $x = 10^{22}$ 時誤差 3.6×10^{87} ULP）：33 位元一段的 $\pi/2$ 只保證 $n < 2^{20}$ 時 $n \cdot (\pi/2)$ 片段精確。這說明範圍縮減才是 `sin` 最困難的部分，也說明為什麼 `glibc` 需要 5 個區間。
- 速度：`glibc sin` 在 $|x| < 2.43$ 約 6.4 ns，Cody–Waite 區間 23 ns，Payne–Hanek 區間 71 ns（前兩者用了 IFUNC 選出的 FMA 版本）；`x87 fsin` 26–48 ns 且不準；`sinl` (long double) 17–179 ns。
- `sinf` 的最大誤差 0.56 ULP、`sinq` (libquadmath) 1.21 ULP、`sinl` 1.40 ULP —— 同一個函數在不同型別的實作精度也不同。

Q3. IEEE 754 與 `-ffast-math`

Q3.1 它做了什麼

`-ffast-math` 告訴 GCC：「我不在乎 IEEE 754 的語意，請把浮點數當實數最佳化」。它等同於開啟下列選項，並定義巨集 `__FAST_MATH__`：

選項	允許編譯器做的事	破壞了什麼
<code>-fno-math-errno</code>	<code>sqrt</code> 等不設定 <code>errno</code> → 可以直接用 <code>sqrtsd</code> 指令、可向量化 <code>libm</code> 呼叫	<code>errno</code> 錯誤回報
<code>-funsafe-math-optimizations = -fassociative-math + -freciprocal-math + -fno-signed-zeros + -fno-trapping-math</code>	$(a+b)+c \rightarrow a+(b+c)$ (可向量化歸約)； $x/y \rightarrow x(1/y)$ ； $x+0.0 \rightarrow x$ ；假設不會有 <code>trap</code>	結合律/捨入順序、正確捨入的除法、 <code>-0</code> 的符號
<code>-ffinite-math-only</code>	假設沒有 NaN 與 $\pm\text{Inf}$	<code>isnan()</code> / <code>isinf()</code> 可被直接判定為 <code>false</code>
<code>-fno-rounding-math</code> , <code>-fno-signaling-nans</code>	假設預設捨入模式、無 signaling NaN	(本來就是 GCC 預設)
<code>-fcx-limited-range</code>	複數除法不做縮放與 NaN 修正	複數除法的溢位與 <code>Inf</code> 處理
<code>-fexcess-precision=fast</code>	允許中間結果保留在較寬的暫存器	在 x87/float16 上結果可能依暫存器分配而變
連結 <code>crtfastmath.o</code>	程式啟動時設定 MXCSR 的 FTZ (flush-to-zero) 與 DAZ (denormals-are-zero)	非正規數全部變成 0 —— 影響整個行程，包括沒用 <code>-ffast-math</code> 編譯的函式庫

(`-Ofast` = `-O3 -ffast-math` + 其他；RAPTOR 的 makefile 就是用 `-Ofast`。)

Q3.2 範例：只因為 `-ffast-math` 而改變的結果 (`experiments/q3_fastmath.cpp`)

同一份原始碼以 `g++ -O3` 與 `g++ -O3 -ffast-math` 各編譯一次；所有輸入來自 `volatile`，避免常數摺疊：

```
bash — q3_fastmath.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/q3_strict
=== build: -O3 ===
MXCSR = 0x1f80 (FTZ=0, DAZ=0) <- crtfastmath.o sets these at program start
[1] contains_nan(v) with v[500] = 0.0/0.0      -> true
    is_inf(1e308 * 10)                        -> true
[2] sum of 1e7 x 0.1f: naive = 1087937.000000   kahan = 1000000.000000   (exact 1000000.0149...)
[3] dot(a, b) over 2^20 terms of mixed magnitude = -6241446.2278898535
[4] add_zero(-0.0) = 0, 1/add_zero(-0.0) = inf
[5] x/3 for 1e6 values: 0 results differ from correctly-rounded x/3 (hash e8248bcc452606bb)
[6] 1e-308 * 1e-5 = 1e-313   (subnormal kept)
    4.9e-324 (smallest subnormal) * 1e300 = 4.94066e-24
[7] sqrt(-1) = -nan, errno = EDOM
[8] dot() speed: 0.828 ns per element (checksum -1.24829e+09)

user@hw2:~/NTHU-AdvCPP-HW2$ build/q3_fast
```

```

bash — q3_fastmath.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/q3_fast
=== build: -O3 -ffast-math ===
MXCSR = 0x9fc0 (FTZ=1, DAZ=1) <- crtfastmath.o sets these at program start
[1] contains nan(v) with v[500] = 0.0/0.0 -> false
    is_inf(1e308 * 10) -> false
[2] sum of 1e7 x 0.1f: naive = 975599.125000 kahan = 975599.125000 (exact 1000000.0149...)
[3] dot(a, b) over 2^20 terms of mixed magnitude = -6241446.2278382154
[4] add_zero(-0.0) = -0, 1/add_zero(-0.0) = -inf
[5] x/3 for 1e6 values: 333328 results differ from correctly-rounded x/3 (hash 4b594d8d126234dd)
    e.g. 1.000003 / 3 -> 0.333343333333329 (correctly rounded: 0.333343333333334)
[6] 1e-308 * 1e-5 = 0 (subnormal flushed to zero)
    4.9e-324 (smallest subnormal) * 1e300 = 0
[7] sqrt(-1) = -nan, errno = 0
[8] dot() speed: 0.589 ns per element (checksum -1.24829e+09)

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/kerr_render_03 --type double --res 192 --disk --out build/fm_03_double.bin

```

#	實驗	-O3	-O3 - ffast-math	原因
1	陣列中有 0.0/0.0 產生的 NaN, std::isnan 檢查	true	false	-ffinite-math-only : isnan 被最佳化成常數 false —— 錯誤檢查整段消失
1b	std::isinf(1e308 * 10)	true	false	同上
2	10 ⁷ 個 0.1f 加總: naive / Kahan	1087937 / 1000000	975599 / 975599	-fassociative-math 認為 Kahan 的補償項 c = (t - s) - y 代數上恆為 0, 把它刪掉, Kahan 退化成普通加總; naive 加總也因為向量化改變順序而得到不同的錯誤結果
3	2 ²⁰ 項不同數量級的內積	...8898535	...8382154	歸約被重新結合成 SIMD 的部分和, 捨入順序不同
4	1/(-0.0 + 0.0)	+inf	-inf	-fno-signed-zeros : x + 0.0 → x, 但 -0 + +0 在 IEEE 是 +0
5	10 ⁶ 個 x/3	0 個與正確捨入不同	333328 個不同	-freciprocal-math : x/3 → x*(1/3), 兩次捨入, 約 1/3 的結果差 1 ulp
6	1e-308 × 1e-5 (應為非正規數 1e-313)	1e-313	0	crtfastmath.o 設了 FTZ/DAZ (MXCSR 0x1f80 → 0x9fc0)
7	sqrt(-1) 的 errno	EDOM	0	-fno-math-errno
8	內積速度	0.83 ns/元素	0.59 ns/元素	允許向量化歸約 (1.4× 快)

最典型的單一例子是 #2 : Kahan summation 是教科書上「修正浮點誤差」的演算法, 而 -ffast-math 會默默把它最佳化掉 —— 因為在實數的世界裡它什麼都沒做。#1 則是最危險的: 程式中防呆的 NaN 檢查會消失, 錯誤在下游才以奇怪的形式出現。

Q3.3 真實程式: Kerr 引擎與 RAPTOR

```

bash — q3_fastmath.txt

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/kerr_render_03 --type double --res 192 --disk --out build/fm_03_double.bin
type=double method=rk4 spin=0.9375 incl=60 res=192 time=3.094 s (11914 rays/s, 175.6 ns/step)
captured=666 escaped=18383 disk=17815 maxsteps=0 notfinite=0 mean steps=478.0 flux=8.0813024942e+00

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/kerr_render_03 --type float --res 192 --disk --out build/fm_03_float.bin
type=float method=rk4 spin=0.9375 incl=60 res=192 time=2.972 s (12402 rays/s, 166.7 ns/step)
captured=666 escaped=18383 disk=17815 maxsteps=0 notfinite=0 mean steps=478.0 flux=8.0813019326e+00

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/kerr_render_fast --type double --res 192 --disk --out build/fm_fast_double.bin
type=double method=rk4 spin=0.9375 incl=60 res=192 time=3.052 s (12080 rays/s, 173.2 ns/step)
captured=666 escaped=18383 disk=17815 maxsteps=0 notfinite=0 mean steps=478.0 flux=8.0813024942e+00

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/kerr_render_fast --type float --res 192 --disk --out build/fm_fast_float.bin
type=float method=rk4 spin=0.9375 incl=60 res=192 time=2.940 s (12540 rays/s, 166.8 ns/step)
captured=666 escaped=18383 disk=17815 maxsteps=0 notfinite=0 mean steps=478.0 flux=8.0813027737e+00

user@hw2:~/NTHU-AdvCPP-HW2$ python3 scripts/compare.py king build/fm_03_double.bin build/fm_fast_double.bin -O3 -O3 -ffast-math
-O3 vs -O3 -ffast-math: 36864 pixels, status differs 0, intensity bitwise differs 14833 (40.2%), final theta differs 23713, max rel. intensity diff 2.357e-13, flux 8.0813024942104779 vs 8.0813024942104761

```

- 本引擎 (192×192, 含吸積盤) : double 版本有 40% 的像素逐位元改變 (最大相對差 2.4×10^{-13} , 總通量在第 16 位不同) ; float 版本 43% 的像素改變 (最大相對 1.3×10^{-4} 、總通量差 10^{-7}) ; 捕獲/逃逸分類都沒有改變。速度差異 < 2% —— 這個程式的瓶頸是相依鏈與 `sin/cos` , `-ffast-math` 幾乎沒有好處, 卻讓結果不可重現。
- RAPTOR 預設的 `-Ofast` vs 符合 IEEE 的 `-O3` (完整 GRMHD 影像 500×500, dump040, 100 GHz) : 總通量印出的 7 位數相同, 但有 30 個像素在輸出的 6 位有效數字上不同, 最大相對差異 3.7% (都在少數貼近光子環或低強度的像素) ; 執行時間 69 s vs 72 s , 只快 4%。以光線層級比較 (`raptor_compare_ofast`) , `-Ofast` 下 RAPTOR 的測地線統計數字與 `-O2` 完全相同。

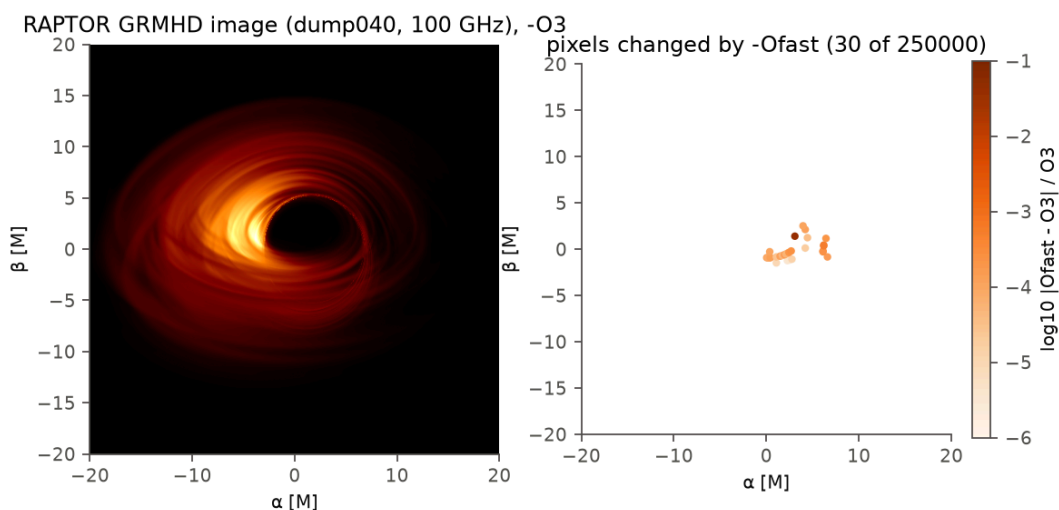


圖 7 : 左 : RAPTOR 產生的 GRMHD 影像 (-O3) ; 右 : -Ofast 改變的像素位置與相對大小。

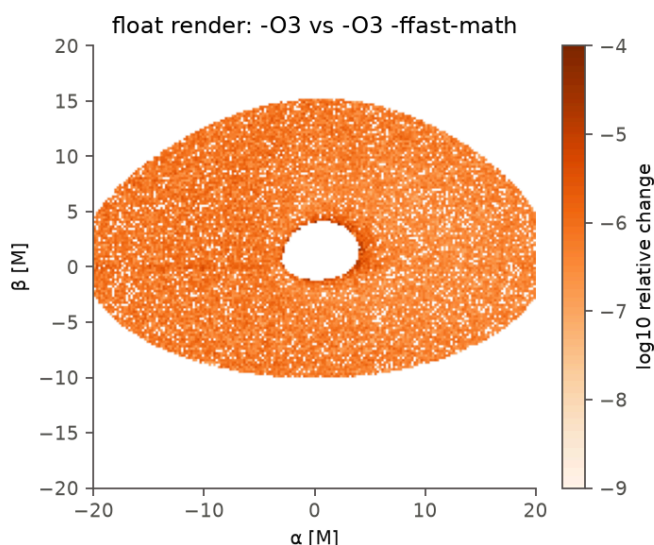


圖 8 : 本引擎 float 版本在 -ffast-math 下改變的像素 (相對差異, 對數色階)。

Q3.4 建議

- 不要對整個專案開 `-ffast-math` ; 它會關掉 NaN 檢查、破壞補償演算法, 並透過 FTZ/DAZ 影響同一行程中的其他程式碼。
- 需要速度時只開真正需要的子選項 : `-fno-math-errno` (幾乎總是安全, 讓 `sqrt` 內聯)、`-fno-trapping-math` ; 需要向量化歸約時自己寫多個部分和 (如 `experiments/cache_sweep.cpp`) , 而不是讓編譯器重新結合。
- 如果一定要用, 對關鍵函式以 `__attribute__((optimize("no-fast-math")))` 或把它放到獨立的編譯單元。

Q5. (加分) 相同程式碼、相同編譯器版本、相同參數、無 UB，結果卻不同

experiments/q5_env.c 用同一份原始碼、同為 GCC 13.3.0、同樣只加 `-O2` 編出三個版本：x86-64 + glibc 2.39、x86-64 + musl 1.2.4 (musl-gcc 只是包裝同一個 gcc)、aarch64 + glibc 2.39 (aarch64-linux-gnu-gcc，以 QEMU 執行)。程式沒有未定義行為，所有 libm 的輸入以 `volatile` 強制捨入，保證三個平台餵給函式庫的位元完全相同。

```
bash — q5_env.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/q5_x86_glibc build/q5_x86_glibc.bin
platform: x86-64 / glibc 2.39, sizeof(long double)=16, LDBL_MANT_DIG=64, FLT_EVAL_METHOD=0
[1] 0.1*10 - 1 = 0
    b*b-4ac (double root) = 0 -> sqrt = 0
[2] (1.0L + 1e-17L) - 1.0L = 9.97466e-18
    sum of ten 0.1L == 1.0L ? no (diff 1.08e-19)
[3] libm hashes over 1000000 deterministic inputs:
    sin=929b cos=a010 tan=0459 exp=a79e log=75c0 pow=4357 atan2=ccc5 cbrt=1e75 sinh=914f lgamma=bc45

user@hw2:~/NTHU-AdvCPP-HW2$ env GLIBC_TUNABLES=glibc.cpu.hwcaps=-AVX2,-FMA build/q5_x86_glibc build/q5_x86_glibc_nofma.bin
platform: x86-64 / glibc 2.39, sizeof(long double)=16, LDBL_MANT_DIG=64, FLT_EVAL_METHOD=0
[1] 0.1*10 - 1 = 0
    b*b-4ac (double root) = 0 -> sqrt = 0
[2] (1.0L + 1e-17L) - 1.0L = 9.97466e-18
    sum of ten 0.1L == 1.0L ? no (diff 1.08e-19)
[3] libm hashes over 1000000 deterministic inputs:
    sin=52f1 cos=7867 tan=63db exp=f2f9 log=3d4f pow=579d atan2=fa88 cbrt=1e75 sinh=a571 lgamma=8a17

user@hw2:~/NTHU-AdvCPP-HW2$ build/q5_x86_musl build/q5_x86_musl.bin
platform: x86-64 / musl, sizeof(long double)=16, LDBL_MANT_DIG=64, FLT_EVAL_METHOD=0
[1] 0.1*10 - 1 = 0
    b*b-4ac (double root) = 0 -> sqrt = 0
[2] (1.0L + 1e-17L) - 1.0L = 9.97466e-18
    sum of ten 0.1L == 1.0L ? no (diff 1.08e-19)
[3] libm hashes over 1000000 deterministic inputs:
    sin=d58a cos=b07f tan=3430 exp=f2f9 log=3d4f pow=579d atan2=0238 cbrt=c637 sinh=3704 lgamma=8a17

user@hw2:~/NTHU-AdvCPP-HW2$ qemu-aarch64 -L /usr/aarch64-linux-gnu build/q5_arm_glibc build/q5_arm_glibc.bin
platform: aarch64 / glibc 2.39, sizeof(long double)=16, LDBL_MANT_DIG=113, FLT_EVAL_METHOD=0
[1] 0.1*10 - 1 = 5.5511151231257827e-17
    b*b-4ac (double root) = 8.8817841970012602e-18 -> sqrt = 2.98023e-09
[2] (1.0L + 1e-17L) - 1.0L = 1e-17
    sum of ten 0.1L == 1.0L ? no (diff -9.63e-35)
[3] libm hashes over 1000000 deterministic inputs:
    sin=929b cos=a010 tan=0459 exp=a79e log=75c0 pow=4357 atan2=ccc5 cbrt=88ab sinh=914f lgamma=4dbd

user@hw2:~/NTHU-AdvCPP-HW2$ python3 scripts/compare.py libm build/q5_x86_glibc.bin build/q5_x86_glibc_nofma.bin glibc(FMA ifunc) glibc(-AVX2,-FMA)
libm results that differ between glibc(FMA ifunc) and glibc(-AVX2,-FMA) (1000000 inputs each):
sin      686 differ ( 0.069%) max 1 ulp
cos      695 differ ( 0.069%) max 1 ulp
```

例一：硬體 (ISA) 不同 → FMA 自動融合 (contraction)

`double mul_sub(a,b,c) { return a*b - c; }`，`0.1*10 - 1`：x86-64 印出 0、aarch64 印出 5.55×10^{-17} 。原因：GCC 在 GNU 模式預設 `-ffp-contract=fast`，只要目標指令集有 FMA 就把 `a*b-c` 融合成一條 FMA (只捨入一次)。aarch64 的基本指令集就有 FMA (反組譯為 `fnmsub d0,d0,d1,d2`)，x86-64 的基本指令集 (SSE2) 沒有，所以是 `mulsd` + `subsd` (兩次捨入：`0.1*10` 先捨入成剛好 1.0)。C/C++ 標準允許這種 contraction (C 6.5p8、FP_CONTRACT)，所以不是 UB。後果不只最後一位：二次方程式重根的判別式 b^2-4ac 在 x86-64 為 0 (一個重根)，在 aarch64 為 8.9×10^{-18} ，開根號得到 3×10^{-9} —— 重根變成兩個不同的根。

同樣的原因，我的 Kerr 引擎在 aarch64 上 (同樣 `-O2`) 有 $4054/9216 = 44\%$ 的像素與 x86-64 逐位元不同 (最大相對差 5.5×10^{-14} ，分類結果相同)。

例二：ABI 不同 → `long double` 是不同的型別

`(1.0L + 1e-17L) - 1.0L`：x86-64 為 9.97×10^{-18} (80-bit，64 位尾數)、aarch64 為 1×10^{-17} (binary128，113 位尾數)；十個 0.1L 相加與 1.0L 的差： 1.08×10^{-19} vs -9.6×10^{-35} 。(MSVC 上 `long double` 就是 `double`，又會是第三種答案。)

例三：作業系統 / C 函式庫不同 → libm 不同

C 標準不要求 `sin`、`exp` 正確捨入 (Q2)，各家實作的演算法不同。同樣是 x86-64、相同輸入，glibc 與 musl 的結果：

```
bash — q5_env.txt
user@hw2:~/NTHU-AdvCPP-HW2$ python3 scripts/compare.py libm build/q5_x86_glibc.bin build/q5_x86_glibc_nofma.bin glibc(FMA ifunc) glibc(-AVX2,-FMA)
libm results that differ between glibc(FMA ifunc) and glibc(-AVX2,-FMA) (1000000 inputs each):
sin      686 differ ( 0.069%) max 1 ulp
cos      695 differ ( 0.069%) max 1 ulp
tan       10 differ ( 0.001%) max 1 ulp
exp      611 differ ( 0.061%) max 1 ulp
log        1 differ ( 0.000%) max 1 ulp
pow      726 differ ( 0.073%) max 1 ulp
atan2    279 differ ( 0.028%) max 1 ulp
cbrt       0 differ ( 0.000%) max 0 ulp
sinh     613 differ ( 0.061%) max 2 ulp
lgamma     8 differ ( 0.001%) max 2 ulp

user@hw2:~/NTHU-AdvCPP-HW2$ python3 scripts/compare.py libm build/q5_x86_glibc.bin build/q5_x86_musl.bin x86-64 glibc x86-64 musl
libm results that differ between x86-64 glibc and x86-64 musl (1000000 inputs each):
sin     31377 differ ( 3.138%) max 1 ulp
cos     31031 differ ( 3.103%) max 1 ulp
tan     35615 differ ( 3.562%) max 1 ulp
exp      611 differ ( 0.061%) max 1 ulp
log        1 differ ( 0.000%) max 1 ulp
pow      726 differ ( 0.073%) max 1 ulp
atan2   177733 differ (17.773%) max 1 ulp
cbrt   496976 differ (49.698%) max 3 ulp
sinh      813 differ ( 0.081%) max 2 ulp
lgamma     8 differ ( 0.001%) max 2 ulp

user@hw2:~/NTHU-AdvCPP-HW2$ python3 scripts/compare.py libm build/q5_x86_glibc.bin build/q5_arm_glibc.bin x86-64 glibc aarch64 glibc
libm results that differ between x86-64 glibc and aarch64 glibc (1000000 inputs each):
sin        0 differ ( 0.000%) max 0 ulp
cos        0 differ ( 0.000%) max 0 ulp
tan        0 differ ( 0.000%) max 0 ulp
exp        0 differ ( 0.000%) max 0 ulp
log        0 differ ( 0.000%) max 0 ulp
pow        0 differ ( 0.000%) max 0 ulp
atan2      0 differ ( 0.000%) max 0 ulp
cbrt    189022 differ (18.902%) max 4 ulp
sinh        0 differ ( 0.000%) max 0 ulp
lgamma   231694 differ (23.169%) max 1 ulp

user@hw2:~/NTHU-AdvCPP-HW2$ build/kerr_render --res 96 --disk --out build/q5_kerr_x86.bin
```

musl 的 `sin/cos/tan` 源自 `fdlibm`，約 3% 的結果與 glibc 差 1 ulp；`atan2` 18%、`cbrt` 50% (最大 3 ulp)。有趣的是 musl 的 `exp/log/pow` 與 glibc 的差異，和 glibc「FMA 版 vs 非 FMA 版」的差異完全相同 (611/1/726 個) —— 兩者用的是同一份演算法 (ARM optimized-routines)，差別只在有沒有 FMA。aarch64 的 glibc 與 x86-64 的 glibc (FMA 版) 在 `sin/exp/pow` 等完全一致，但 `cbrt` (19%，最大 4 ulp) 與 `lgamma` (23%) 不同 —— 這兩個函數在 x86-64 沒有 FMA 變體 (遮掉 FMA 時 `cbrt` 0 個不同)，推測是 aarch64 的 libm 在編譯時被自動 FMA 融合 (例一的機制)，而 x86-64 基本版是分開的乘、加。

例四：同一個 binary、同一個 glibc，只是 CPU 不同

glibc 在執行期依 CPU 功能以 IFUNC 選擇 `sin/cos/exp/log/pow/atan2`... 的 FMA 版本 (Q2.2)。在同一台機器上用 `GLIBC_TUNABLES=glibc.cpu.hwcaps=-AVX2,-FMA` 模擬「沒有 FMA 的舊 CPU」：`sin` 有 686 個、`exp` 611、`pow` 726 個 (約 0.07%) 結果不同 (1–2 ulp)。也就是說：同一個執行檔，在 Haswell 之前與之後的 Intel CPU 上會算出不同的 `sin`。

例五：核心數不同 → 平行歸約的結果不同

`#pragma omp parallel for reduction(+:s)` 的預設執行緒數 = CPU 核心數，每個執行緒加總自己的區段後再合併，捨入順序取決於執行緒數：

```
bash — q5_env.txt

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=1 build/q5_threads
threads=1 parallel sum = 2409164.5612163064 serial sum = 2409164.5612163064 diff = 0

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=2 build/q5_threads
threads=2 parallel sum = 2409164.5612166366 serial sum = 2409164.5612163064 diff = 3.3e-07

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=3 build/q5_threads
threads=3 parallel sum = 2409164.5612161369 serial sum = 2409164.5612163064 diff = -1.7e-07

user@hw2:~/NTHU-AdvCPP-HW2$ env OMP_NUM_THREADS=4 build/q5_threads
threads=4 parallel sum = 2409164.5612164908 serial sum = 2409164.5612163064 diff = 1.84e-07
```

1、2、3、4 個執行緒得到 4 個不同的總和（差異 $\sim 10^{-7}$ ，相對 10^{-13} ）。程式完全合法，但在 8 核和 64 核的機器上輸出不同。更進一步：4 執行緒的結果在兩次執行間也不同（本報告開發時一次得到 $\cdots 164913$ 、`run_all.sh` 中得到 $\cdots 164908$ ）—— `libgomp` 以執行緒完成的先後順序合併部分和，所以連同一台機器都不保證可重現。

如何得到可重現的結果：`-ffp-contract=off`（或只在需要時使用 `std::fma`）、不用 `long double`、使用正確捨入的數學函式庫（如 `CORE-MATH`）或自帶實作、固定歸約順序（固定分塊數而非依執行緒數分塊，或用 Kahan/exact accumulator）、在測試中以誤差容忍度而不是逐位元相等比較結果。

Q6. 總結

Q6.1 我對浮點數運算的看法

浮點數不是「不安全」，而是一個規格嚴謹的近似系統：IEEE 754 保證基本運算正確捨入，所以單一運算的誤差 $\leq \frac{1}{2} \text{ulp}$ ，完全可預測。問題出在我們把它當成實數：

1. 誤差會累積與放大：抵銷（相近數相減）、長鏈加總、病態問題都會放大捨入誤差 —— 需要分析或實驗量化，而不是祈禱。
2. 「同樣的程式」不保證同樣的位元：編譯器（contraction、`-ffast-math`、向量化）、函式庫（libm 精度是 implementation-defined）、硬體（FMA dispatch、`long double` 格式）、平行化（歸約順序）都會改變結果（Q3、Q5）。
3. 通常截斷誤差比捨入誤差大：本引擎中 RK4 的截斷誤差 $\sim 10^{-8}$ – 10^{-9} ，double 的捨入誤差 $\sim 10^{-15}$ ；RAPTOR 預設 RK2 的誤差甚至到 10^{-3} 。先確保演算法收斂，再談型別。
4. 驗證手段：與解析解比較（陰影邊界）、收斂階數檢查（ h^4 ）、守恆量監控（E、L、Q、H）、以更高精度重跑（`__float128` 參考解）、與獨立實作（RAPTOR）交叉比對。

實務準則：預設 `double`；`float` 只在可向量化/受頻寬限制時使用；不用 `==` 比較浮點數（用相對+絕對容忍度）；不全域開 `-ffast-math`；小心 NaN/Inf 的傳播並在邊界檢查；輸出十進位時知道 `printf` 在做什麼；需要可重現時固定 contraction、libm 與歸約順序。

Q6.2 額外細節：問題本身的「條件數」—— 再多位數也救不了病態問題

前面的題目都在談「計算」的誤差，但有一種誤差來源與浮點數格式無關：問題本身對輸入的敏感度（condition number）。黑洞陰影邊緣附近的光線會繞光子環好幾圈才離開，光子軌道是不穩定的：每繞半圈，擾動被放大 e^γ 倍（ γ 為 Lyapunov 指數）。`experiments/q6_extra.cpp` 讓兩條光線的撞擊參數只差 10^{-13} （相對），量最終方向的差：

```

bash — q6_extra.txt

user@hw2:~/NTHU-AdvCPP-HW2$ build/q6_extra
A. Conditioning near the photon ring (a=0.9375, i=60, psi=0.3 rad, rho_c=6.562917601581)
rays at rho = rho_c (1 + delta) and rho (1 + 1e-13); angle between final directions
delta      steps      d(direction)      amplification
1e-01      977        3.003e-10          3.003e+03
1e-02      1278       1.393e-10          1.393e+03
1e-03      1473       3.591e-10          3.591e+03
1e-04      1636       1.108e-09          1.108e+04
1e-05      1847       9.198e-09          9.198e+04
1e-06      2146       7.109e-08          7.109e+05
1e-07      2346       7.022e-07          7.022e+06
1e-08      2507       8.095e-06          8.095e+07
1e-09      2712       8.232e-05          8.232e+08
1e-10      3014       7.363e-04          7.363e+09

B. Catastrophic cancellation in the camera set-up (float, i = 60 deg)
k_theta = sqrt(Q - L^2 cot^2 i + cos^2 i) with Q = beta^2 + cos^2 i (alpha^2 - 1), L = -alpha sin i
alpha      beta      RAPTOR formula      exact (=beta)      rel. error
0.5         3         3                   3                   0.00e+00
5           3         3.00000024         3                   7.95e-08
20          3         3.00000262         3                   8.74e-07
20          0.3       0.300019264        0.300000012        6.42e-05
20          0.03      0.0302571896       0.0299999993       8.57e-03
20          0.003     0.00478104409      0.00300000003      5.94e-01

C. Subnormal operands (x *= 0.999999 in a dependent chain, 1e7 iterations)
start 1      (normal ) : 2.76 ns per multiply
start 1e-300 (normal ) : 3.10 ns per multiply

```

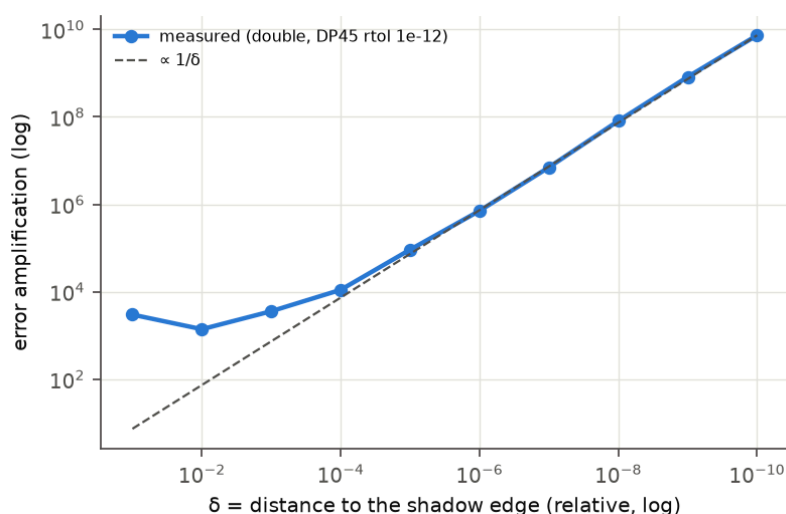


圖 10：誤差放大倍率 $\propto 1/\delta$ (δ = 與陰影邊緣的相對距離)。 $\delta = 10^{-10}$ 時放大 7×10^9 倍。

在 $\delta = 10^{-10}$ 時，輸入 1 ulp 等級的差異 (10^{-16}) 會變成 10^{-6} rad 的方向差； δ 再小，double 的捨入誤差就足以讓「捕獲或逃逸」的結論翻轉。這不是 bug，也不能靠換 **long double** 解決（只能把牆往後推幾位數），而是物理問題本身的性質——這也解釋了第 1.4 節中 RAPTOR 與本引擎的最大誤差為什麼都集中在陰影邊緣。正確的做法是：辨識病態區域、在那裡以統計量（例如像素平均通量）而非單一光線來下結論。

同一個程式中另外兩個小陷阱（同樣在 `results/q6_extra.txt`）：

- 代數等價 $\neq$ 數值等價 (catastrophic cancellation)：RAPTOR 設定光線時計算 $k_\theta = \sqrt{Q - L^2 \cot^2 i + \cos^2 i}$ ，代入後代數上恆等於 β ，但當 $|\alpha| \gg |\beta|$ 時是大數相減：float 下 $\alpha = 20$ 、 $\beta = 0.003$ 時算出 0.00478，相對誤差 59%。直接寫 $k_\theta = \beta$ 就沒有這個問題（引擎提供 `exact_ktheta` 選項）。

- 非正規數很慢：相依乘法鏈中運算元為非正規數時，每次乘法 45.7 ns，正常值 2.8 ns，慢 17 倍 (microcode assist) —— 這也是 `-ffast-math` 設定 FTZ/DAZ 的動機，但代價是 Q3 的 #6。

Q7. AI 使用自我揭露

(請依實際情況修改本段。)

本作業使用 Anthropic 的 AI 程式助理 Claude (Claude Code) 協助。

- AI 完成的部分：依我的指示（「寫一個 Kerr 度規測地線模擬引擎並與 RAPTOR 比較、加入平行化技術與快取命中率的比較」）撰寫了引擎程式碼、RAPTOR 橋接與比對程式、所有實驗程式、執行腳本、圖表、截圖工具與本報告初稿；並查閱了 glibc 2.39 的 `s_sin.c` 原始碼作為 Q2 的依據。
- 我自己做的部分：（請填寫，例如：決定報告主題與實驗方向、閱讀並理解每一個實驗與程式碼、在自己的電腦上重跑 `run_all.sh` 核對數據、修改與補充說明、最終審閱。）
- 所有數據都來自實際執行的輸出（`results/*.txt`），截圖是把這些終端輸出以終端機樣式渲染成圖片（`scripts/screenshots.js`）。